

LAPORAN PRAKTIKUM
STRUKTUR DATA
PEKAN KE 6

Disusun Oleh :

Muhammad Irfan 2511531008

Dosen Pengampu : Dr Wahyudi S.T M.T

Asisten Praktikum : Muhammad Galid Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2025

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum dari mata kuliah Struktur Data dengan judul Double Linked List

Laporan ini disusun sebagai bentuk hasil kegiatan praktikum Struktur Data yang di mana membahas tentang bahasa pemrograman java dan penggunaan Double linked list serta kegunaan masing-masing percabangan tersebut serta cara penerapannya.

Penulis menyadari bahwa laporan praktikum ini masih jauh dari sempurna baik dari segi penulisannya dan dari segi pembahasannya. Oleh karena itu saya ingin mengucapkan terima kasih kepada dosen pengampu, asisten praktikum yang telah membimbing serta rekan-rekan yang telah membantu dalam proses penyusunan laporan ini.

Akhir kata, penulis ucapkan terima kasih pada dosen pengampu yang telah memberikan arahan dan semoga laporan ini dapat bermanfaat baik bagi penulis maupun pembaca dalam pemahaman tentang Double linked list.

DAFTAR ISI

DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	2
1.Class NodeDLL_2511531008	2
Kode program	2
2. Class InsertDLL_2511531008	2
Kode program	2
Output	5
3.Class PenelusuranDLL_2511531008.....	5
Kode program	5
Ouput.....	7
4. HapusDLL_2511531008.....	7
kodeprogram	7
Output	10
BAB III PENUTUP	11
3.1 Kesimpulan	11
TUGAS PEKAN 4.....	12
1.Class Pasien_2511531008	12
2. Class RumahSakit_251153108	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan komponen yang paling penting untuk menyimpan nilai dan mengelola nilai secara baik dan efisien. Penyimpanan nilai dan data sangatlah penting dalam dunia pemrograman. Penyimpanan data array sangatlah terbatas. Ukurannya yang statis dan tidak bisa berkelanjutan membuat program menjadi lambat dan tidak efisien

Oleh karena itu, dibuatkan cara penyimpanan dan pengelolaan data dengan baik dengan benar. Salah satunya dengan double linked list. Double linked list merupakan struktur data yang menyimpan nilai dari node yang saling terhubung dalam satu arah. Setiap node menyimpan 1 nilai/data dan memiliki pointer ke node berikutnya. Hal ini penggunaan memori yang lebih fleksibel tanpa merombak seluruh struktur memori.

1.2 Tujuan Praktikum

Tujuan utama dari praktikum ini adalah memberikan pemahaman kepada mahasiswa tentang Konsep Double linked list serta implementasi di dalam pemrograman bahasa Java. Selain itu bagaimana pengaplikasian berbagai berbagai-macam problem dengan menggunakan Double linked list

1.3 Manfaat Praktikum

Manfaat utama dari praktikum ini adalah mahasiswa diharapkan dapat memahami konsep, operasi operasi dan mengimplementasikan dalam bahasa java. Selain itu, mahasiswa juga diharapkan untuk dapat mengolah dan meningkatkan analisis terhadap problem dengan menggunakan Double linked list.

BAB II

PEMBAHASAN

1. Class NodeDLL_2511531008

Kode program

```
1 package pekan6_2511531008;
2
3 public class NodeDLL_2511531008 {
4     int data_1008;
5     NodeDLL_2511531008 next_1008;
6     NodeDLL_2511531008 prev_1008;
7
8     public NodeDLL_2511531008(int data_1008) {
9         this.data_1008 = data_1008;
10        this.prev_1008 = prev_1008;
11        this.next_1008 = next_1008;
12    }
13 }
14
```

Penjelasan :

- Pembuatan kelas NodeDLL_2511531008 dengan atribut terdiri dari data, next, prev dan attribute node
- Konstruktor node terdiri dari attribute data_1008 dan juga next_1008
- Next_1008 berfungsi sebagai pointer ke node setelahnya.
- prev_1008 berfungsi sebagai pointer ke node sebelumnya

2. Class InsertDLL_2511531008

Kode program

```
1 package pekan6_2511531008;
2
3 public class InsertDLL_2511531008 {
4     static NodeDLL_2511531008 insertBegin_1008(NodeDLL_2511531008 head, int data) {
5         // buat node baru
6         NodeDLL_2511531008 new_node = new NodeDLL_2511531008(data);
7         //jadikan pointer nextnya head
8         new_node.next_1008 = head;
9         // jadikan pointer prev head ke new_node
10        if(head != null) {
11            head.prev_1008 = new_node;
12        }
13        return new_node;
14    }
15 }
```

Penjelasan :

- Membuat function insertBegin_1008 berguna untuk menyisipkan data ke node pertama pada DLL
- Diawali dengan membuat node, kemudian buat pointer nextnya head, previous head dari nwnode
- Jika head berisi maka jadikan newNode_1008 sebagai pointer prev

```

16 public static NodeDLL_2511531008 insertEnd_1008(NodeDLL_2511531008 head_1008, int NewData_1008) {
17     //buat node baru
18     NodeDLL_2511531008 newNode_1008 = new NodeDLL_2511531008(NewData_1008);
19     //jika dll null jadikan head
20     if(head_1008 == null) {
21         head_1008 = newNode_1008;
22     }
23     else {
24         NodeDLL_2511531008 curr_1008 = head_1008;
25         while(curr_1008.next_1008 != null) {
26             curr_1008 = curr_1008.next_1008;
27         }
28         curr_1008.next_1008 = newNode_1008;
29         newNode_1008.prev_1008 = curr_1008;
30     }
31     return head_1008;
32 }

```

Penjelasan :

- Diawali dengan membuat objek yang bernama new_node_1008
- Jika head_1008 kosong, maka jadikan nwNode_1008 sebagai headnya
- Jika tidak, maka lakukan perulangan hingga menemukan node terakhirnya atau node yang tidak mempunyai pointer alias null
- Lakukan penyambungan node terakhir dengan node baru. Prev node baru letakkan pada node terakhir. Sedangkan next node terakhir letakkan pada node baru

```

33 public static NodeDLL_2511531008 insertAtPosition(NodeDLL_2511531008 head_1008, int pos_1008, int new_data_1008) {
34     //buat node baru
35     NodeDLL_2511531008 new_node_1008 = new NodeDLL_2511531008(new_data_1008);
36     if(pos_1008 == 1) {
37         new_node_1008.next_1008 = head_1008;
38         if(head_1008 == null) {
39             head_1008.prev_1008 = new_node_1008;
40         }
41         head_1008 = new_node_1008;
42         return head_1008;
43     }
44     NodeDLL_2511531008 curr_1008 = head_1008;
45     for(int i = 1; i < pos_1008- 1 && curr_1008 != null;++i) {
46         curr_1008 = curr_1008.next_1008;
47     }
48     if(curr_1008 == null) {
49         System.out.print("posisi tidak ada");
50         return head_1008;
51     }
52     new_node_1008.prev_1008= curr_1008;
53     new node 1008.next 1008 = curr 1008.next 1008;

54     curr_1008.next_1008 = new_node_1008;
55     if(new_node_1008.next_1008 != null) {
56         new_node_1008.next_1008.prev_1008 = new_node_1008;
57     }
58     return head_1008;

```

Penjelasan:

- Jika pos_1008 == 1, maka lakukan seperti di fungsi innsertBegin sebelumnya.
- Jika tidak, lakukan perulangan for hingga berjalan ke posisi yang tepat tepat sebelum lokasi penempatan
- New_node prev disambungkan ke curr, new_node next disambungkan ke node setelah curr. Curr. Diarahkan ke new_node
- Jika ada node setelah new_node, maka prev pada node tersebut harus kembali ke new_node

```

63 public static void printList_1008(NodeDLL_2511531008 head_1008) {
64     NodeDLL_2511531008 curr_1008 = head_1008;
65     while(curr_1008 != null) {
66         System.out.print(curr_1008.data_1008 + " <-> ");
67         curr_1008 = curr_1008.next_1008;
68     }
69     System.out.println();
70 }

```

Penjelasan:

- Fungsi printList ini mencetak semua nilai/list dengan simbol "<->". Printlist ini membantu kita dalam memvisualisaikan bentuk dari double linked list

```

71 public static void main(String[] args) {
72     //membuat ddl 2 <-> 3 <-> 5
73     NodeDLL_2511531008 head_1008 = new NodeDLL_2511531008(2);
74     head_1008.next_1008 = new NodeDLL_2511531008(3);
75     head_1008.next_1008.prev_1008 = head_1008;
76     head_1008.next_1008.next_1008 = new NodeDLL_2511531008(5);
77     head_1008.next_1008.next_1008.prev_1008 = head_1008.next_1008;
78     //cetak DLL awal
79     printList_1008(head_1008);
80     //tambah 1 diawal
81     head_1008 = insertBegin_1008(head_1008, 1);
82     System.out.print("simpul 1 ditambah di awal : ");
83     printList_1008(head_1008);
84     //tambah 6 di akhir
85     System.out.print("simpul 6 ditambah diakhir:");
86     int data_1008 = 6;
87     head_1008 = insertEnd_1008(head_1008, data_1008);
88     printList_1008(head_1008);
89     //menambah node 4 di posisi 4
90     System.out.print("ditambah node 4 di posisi 4: ");
91     int data2_1008 = 4;

```

Penjelasan :

- Baris 73-77 merupakan inisialisasi nilai pada node secara berutan
- Baris 79 mencetak semua nilai pada ddl
- Baris 81-82 menyisipkan nilai di awal node dan mencetak semua node
- Baris 85-86 menyisip nilai di akhir DLL, dan mencetak semua node pada DLL
- Baris 89-90 menyisipkan nilai pada posisi yang spesifik, kemudian cetak semua nilai pada DLL

Output

```

<terminated> TambahSLL_2511531008 [Java Application] C:\Users\rifqi\.p2\po
Senarai berantai awal :2-->3-->5-->6
tambah 1 simpul di depan : 2-->3-->5-->6-->1
tambah 1 simpul di belakang : 2-->3-->5-->6-->1-->7
2-->3-->5-->4-->6-->1-->7

```

3.Class PenelusuranDLL_2511531008

Kode program

```

3 public class PenelusuranDLL_2511531008 {
4     static void forwardTraversal_1008(NodeDLL_2511531008 head_1008) {
5         NodeDLL_2511531008 curr_1008 = head_1008;
6         //lanjutkan sampai akhir
7         while(curr_1008 != null) {
8             //print data
9             System.out.print(curr_1008.data_1008 + " <-> ");
10            //curr_1008.pindah ke node berikutnya
11            curr_1008 = curr_1008.next_1008;
12        }
13        //print spasi
14        System.out.println();
15    }
16 }

```


Penjelasan:

- Fungsi untuk membaca node dari node pertama sampai node yang terakhir
- Diawali dengan membuat objek DLL yang bernama curr_1008, kemudian lakukan perulangan hingga curr bernilai null
- Di dalam looping while, program akan mencetak data, kemudian memindahkan ke pointer berikutnya dengan menggunakan curr_1008.next

```
// Fungsi penelusuran mundur
static void backwardTraversal_1008(NodeDLL_2511531008 tail_1008) {
    //mulai dari akhir
    NodeDLL_2511531008 curr_1008 = tail_1008;
    // lanjut sampai head
    while(curr_1008 != null) {
        //cetak data
        System.out.print(curr_1008.data_1008 + " <-> ");
        //pindah ke node sebelumnya
        curr_1008 = curr_1008.prev_1008;
    }
    // cteka spasi
    System.out.println();
}
```

Penjelasan :

- Fungsi ini mirip dengan yang pertama, perbedaanya fungsi ini membaca dari akhir node hingga node yang pertama yang prinsipnya mirip dengan LIFO.
- Di awali dengan membuat objek curr_1008 yang bernilai tail_1008
- Kemudian , program akan melakukan perulangan hingga curr_1008 bernilai kosong
- Perbedaan dengan forwardtraversal sebelumnya, fungsi ini menggunakan prev_1008 sebagai pointer, sehingga list atau node di baca dari belakang.
- Di dalam looping, program akan mencetak nilai masing masing dari pointer prev.

```
31 public static void main(String[] args) {
32     // cetak DLL
33     NodeDLL_2511531008 head_1008= new NodeDLL_2511531008(1);
34     NodeDLL_2511531008 second_1008 = new NodeDLL_2511531008(2);
35     NodeDLL_2511531008 third_1008 = new NodeDLL_2511531008(3);
36
37     head_1008.next_1008 = second_1008;
38     second_1008.prev_1008 = head_1008;
39     second_1008.next_1008 = third_1008;
40     third_1008.prev_1008 = second_1008;
41
42     System.out.println("Penulusran maju : ");
43     forwardTraversal_1008(head_1008);
44     System.out.println("penelusuran mundur : ");
45     backwardTraversal_1008(third_1008);
46 }
```

Penjelasan :

- Baris 33-35 inisialisasi nilai dari DLL

- Baris 37-40 ialah membuat nilai pointer dari masing masing list/ node, misal node pertama (head) maka pointer next nya adalah node yang kedua.
- Baris 42-45 mencetak nilai dengan menggunakan fungsi forwardtraversal (penelusuran dari depan) backwardtraversal (penelusuran dari belakang).

Ouput

```
<terminated> Penelusuran
Penulusran maju      :
1 <-> 2 <-> 3 <->
penelusuran mundur :
3 <-> 2 <-> 1 <->
```

4. HapusDLL_2511531008

kodeprogram

```
5 public static NodeDLL_2511531008 delHead_1008(NodeDLL_2511531008 head_1008) {
6     if(head_1008 == null) {
7         return null;
8     }
9
10    head_1008 = head_1008.next_1008;
11    if(head_1008 != null) {
12        head_1008.prev_1008 = null;
13    }
14    return head_1008;
15 }
```

Penjelasan :

- Jika head_1008 null, maka balikan nilai null
- Jika tidak maka jadikan head.next sebagai head yang baru, kemudian lakukan pengecekan jika head_1008 tidak kosong, maka head_1008.prev_1008 jadikan kosong. Karena head yang baru tidak boleh memiliki pointer prev

```

16 // fungsi menghapus di akhir
17 public static NodeDLL_2511531008 dellLast_1008(NodeDLL_2511531008 head_1008) {
18     if(head_1008 == null) {
19         return null;
20     }
21     if(head_1008.next_1008 == null) {
22         return null;
23     }
24     NodeDLL_2511531008 curr_1008 = head_1008;
25     while(curr_1008.next_1008 != null) curr_1008 = curr_1008.next_1008;
26     //update pointer previous node
27     if(curr_1008.prev_1008 != null) curr_1008.prev_1008.next_1008 = null;
28     return head_1008;
29 }
30 // fungsi menghapus node posisi tertentu

```

Penjelasan

- Fungsi ini merupakan fungsi untuk menghapus node paling belakang
- Jika head_1008 atau head_next kosong, maka balikan nilai kosong
- Jika tidak maka buat objek DLL dengan nama curr_1008
- Lakukan perulangan while looping. Lakukan perulangan while hingga berada di node terakhir atau curr_1008.next_1008 = null.
- Setelah ketemu node terakhir, pergi ke node sebelumnya (prev) dan mengubah penunjuk next milik node sebelum-akhir tersebut menjadi null.

```

30 // fungsi menghapus node posisi tertentu
31 public static NodeDLL_2511531008 delPos_1008(NodeDLL_2511531008 head_1008, int pos_1008) {
32     // jika DLL kosong
33     if(head_1008 == null) return head_1008;
34     NodeDLL_2511531008 curr_1008 = head_1008;
35     //telusuri sampai ke node yang akan dihapus
36     for(int i = 1; curr_1008 != null && i < pos_1008; ++i) curr_1008 = curr_1008.next_1008;
37
38     //jika posisi tidak ditemukan
39     if(curr_1008 == null) return head_1008;
40     //update pointer
41     if(curr_1008.prev_1008 != null) curr_1008.prev_1008.next_1008 = curr_1008.next_1008;
42     if(curr_1008.next_1008 != null) curr_1008.next_1008.prev_1008 = curr_1008.prev_1008;
43     // jika yang dihapus head
44     if(head_1008 == curr_1008) {
45         head_1008 = curr_1008.next_1008;
46     }
47     return head_1008;
48 }
49 public static void printList(NodeDLL_2511531008 head_1008) {
50     NodeDLL_2511531008 curr_1008 = head_1008;
51     while(curr_1008 != null) {
52         System.out.print(curr_1008.data_1008 + " <-> ");
53         curr_1008 = curr_1008.next_1008;
54     }
55     System.out.println();
56 }

```

Penjelasan :

- Fungsi `delpos_1008` merupakan fungsi untuk menghapus DLL dengan posisi yang spesifik
- Lakukan pencarian posisi yang spesifik dengan for looping
- Jika tidak ditemukan balikan nilai parameter semula
- Jika ditemukan maka ubah pointer `next` dan `prev`, kemudian hapus dan balikan nilai dari DLL
- Fungsi `printlist` merupakan fungsi untuk mencetak node/list DLL

```

public static void printList(NodeSLL_2511531008 head_1008) {
    NodeSLL_2511531008 curr_1008 = head_1008;
    while(curr_1008.next_1008 != null) {
        System.out.print(curr_1008.data_1008 + "-->");
        curr_1008 = curr_1008.next_1008;
        if(curr_1008.next_1008 == null) {
            System.out.print(curr_1008.data_1008);
            System.out.println();
        }
    }
}

```

Penjelasan :

- Fungsi ini mencetak semua nilai – nilai node secara berurutan dengan menggunakan pointer

```

57 public static void main(String[] args) {
58     //buat sebuah DLL
59     NodeDLL_2511531008 head_1008 = new NodeDLL_2511531008(1);
60     head_1008.next_1008 = new NodeDLL_2511531008(2);
61     head_1008.next_1008.prev_1008 = head_1008;
62     head_1008.next_1008.next_1008 = new NodeDLL_2511531008(3);
63     head_1008.next_1008.next_1008.prev_1008 = head_1008.next_1008;
64     head_1008.next_1008.next_1008.next_1008 = new NodeDLL_2511531008(4);
65     head_1008.next_1008.next_1008.next_1008.prev_1008 = head_1008.next_1008.next_1008;
66     head_1008.next_1008.next_1008.next_1008.next_1008 = new NodeDLL_2511531008(5);
67     head_1008.next_1008.next_1008.next_1008.next_1008.prev_1008 = head_1008.next_1008.next_1008.next_1008;
68
69     System.out.print("DLL awal: ");
70     printList(head_1008);
71
72     System.out.print("Setelah head dihapus: ");
73     head_1008 = delHead_1008(head_1008);
74     printList(head_1008);
75
76     System.out.print("Setelah node terakhir dihapus: ");
77     head_1008 = delLast_1008(head_1008);
78     printList(head_1008);
79
80
81     System.out.print("Setelah node ke 2: ");
82     head_1008 = delPos_1008(head_1008, 2);
83
84     printList(head_1008);

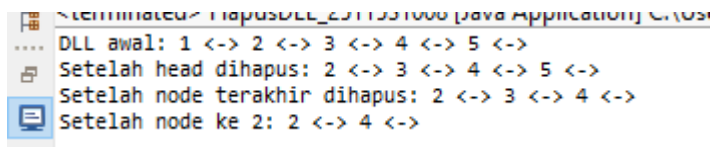
```

Penjelasan :

- Membuat objek double linked list dengan nama `head_1008`

- Kemudian objek head_1008 diisi dengan nilai – nilai seperti 1,2,3,4,5,6
- Baris 69-70 Cetak list node double linked list secara berantai
- Baris 72-74 menggunakan fungsi delHead_1008 yang berfungsi untuk menghapus node dari depan, kemudian cetak
- Baris 76-78 menggunakan fungsi dellast_1008 yang berfungsi untuk menghapus node dari belakang , kemudian cetak nilai setelah di hapus.
- Baris 81-82 menghapus nilai node dengan posisi secara spesifik yaitu posisi yang kedua, kemudian mencetak node DLL setelah di hapus

Output



```

..... DLL awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->
Setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->
Setelah node terakhir dihapus: 2 <-> 3 <-> 4 <->
Setelah node ke 2: 2 <-> 4 <->

```

BAB III

PENUTUP

3.1 Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa double linked list merupakan struktur data linear yang menyimpan nilai dari node node yang saling terhubung dengan pointer next dan previous. Dengan begitu, double linked list lebih efisien daripada array karena lebih fleksibel, hemat memori tanpa merombak struktur penyimpanan memori. Selain itu, terdapat operasi – operasi pada double linked list seperti menyisipkan nilai dari depan, belakang, pencarian data serta menghapus data dari depan, belakang dan posisi spesifik. diharapkan mahasiswa dapat memahami konsep double linked list serta mengimplemtasikan kedalam kehidupan sehari hari.

TUGAS PEKAN 6

1.Class Lagu_2511531008

```
1 package pekan6_2511531008;
2 public class Lagu_2511531008 {
3     private String judul_1008;
4     private String penyanyi_1008;
5     Lagu_2511531008 next_1008; // pointer ke node berikutnya
6     Lagu_2511531008 prev_1008; // pointer ke node sebelumnya
7
8     public Lagu_2511531008(String judul_1008, String penyanyi_1008) {
9         this.judul_1008 = judul_1008;
10        this.penyanyi_1008 = penyanyi_1008;
11        this.next_1008 = null;
12        this.prev_1008 = null;
13    }
14    // getter
15    public String getJudul_1008() {return judul_1008;}
16    public String getPenyanyi_1008() {return penyanyi_1008;}
17    // Setter
18    public void setJudul_1008(String judul_1008) {this.judul_1008 = judul_1008;}
19    public void setPenyanyi_1008(String penyanyi_1008) {this.penyanyi_1008 = penyanyi_1008;}
20 }
21
22 @Override
23 public String toString() {
24     return judul_1008 + " - " + penyanyi_1008;
25 }
```

Penjelasan :

- Class ini membuat konstruktor DLL yang bernama Lagu_2511531008
- Konstruktor yang dibuat terdiri dari judul (string), penyanyi(string) dan 2 pointer yaitu next dan prev
- Atribut judul mempunyai setter dan getter yang berfungsi sebagai pintu masuk dan keluar dari class sehingga aman dan terkontrol.
- Fungsi String toString berfungsi untuk mencetak nilai pada DLL lagu dengan format tertentu

2. Class Musik_2511531008

```
1 package pekan6_2511531008;
2 import java.util.*;
3 public class Musik_2511531008 {
4
5     Lagu_2511531008 head_1008;
6     Lagu_2511531008 tail_1008;
7
8     // 1. tambahLagu_1008() - Menambah lagu baru di AKHIR list
9     public void tambahLagu_1008(String judul, String penyanyi) {
10         Lagu_2511531008 laguBaru_1008 = new Lagu_2511531008(judul, penyanyi);
11         if (head_1008 == null) {
12             head_1008 = laguBaru_1008;
13             tail_1008 = laguBaru_1008;
14         } else {
15             laguBaru_1008.prev_1008 = tail_1008;
16             tail_1008.next_1008 = laguBaru_1008;
17             tail_1008 = laguBaru_1008; // tail diupdate
18         }
19         System.out.println("Lagu berhasil ditambahkan!");
20     }
```

Penjelasan :

- Membuat objek lagu dengan nama head_1008 dan tail_1008
- Method tambahLagu_1008 merupakan menambahkan lagu di akhir list. Jika gak punya head_1008, maka jadikan lagubaru_1008 sebagai head sekaligus dengan tail.
- Jika head_1008 terisi, maka jadikan LaguBaru sebelumnya menjadi tail_1008, dan jadikan next dari lagu sebelumnya menjadi lagu baru

```
21 // 2. hapusLaguAwal_1008() - Menghapus lagu pertama (head)
22 public void hapusLaguAwal_1008() {
23     if (head_1008 == null) {
24         System.out.println("Playlist kosong, tidak ada lagu yang dihapus.");
25         return;
26     }
27     System.out.println(" Lagu \"\" + head_1008.getJudul_1008() + "\" berhasil dihapus.");
28
29     head_1008 = head_1008.next_1008;
30     if (head_1008 != null) {
31         head_1008.prev_1008 = null;
32     }
33 }
34 // 3. tampilLagu_1008() - Tampilkan lagu dari awal ke akhir
```

Penjelasan :

- Fungsi ini menghapus lagi di awal node
- Jika head_1008 kosong, maka tidak ada yang bisa dihapus
- Jika tidak, maka cetak dulu getjudul yang akan dihapus, setelah itu baru dihapus dengan memindahkan pointer


```

34 // 3. tampilMaju_1008() - Tampilkan lagu dari awal ke akhir
35 public void tampilMaju_1008() {
36     if (head_1008 == null) {
37         System.out.println("Playlist kosong.");
38         return;
39     }
40     System.out.println("\n Playlist (Maju :)");
41     System.out.println("-----");
42     Lagu_2511531008 curr_1008 = head_1008;
43     //lanjutkan sampai akhir
44     int nomor = 1;
45     while (curr_1008 != null) {
46         System.out.println(nomor++ + ". " + curr_1008);
47         curr_1008 = curr_1008.next_1008;
48     }
49     //print spasi
50     System.out.println();
51     System.out.println("-----");
52 }

```

Penjelasan:

- Fungsi ini mencetak nilai node dari pertama sampai terakhir
- Untuk mencetak semua node, dibutuhkan perulangan hingga cur_1008 bernilai null atau cur_1008 berada di node terakhir
- Setiap perulangan, program akan mencetak masing masing dari nilai pointer next.

Penjelasan:

```

53 // 4. tampilMundur_1008() - Tampilkan lagu dari akhir ke awal
54 public void tampilMundur_1008() {
55     if (tail_1008 == null) {
56         System.out.println("Playlist kosong.");
57         return;
58     }
59     System.out.println("\n Playlist (Mundur :)");
60     System.out.println("-----");
61     Lagu_2511531008 curr_1008 = tail_1008;
62     int nomor = 1;
63     // traversal mundur menggunakan pointer prev
64     while (curr_1008 != null) {
65         System.out.println(nomor++ + ". " + curr_1008); // otomatis panggil toString()
66         curr_1008 = curr_1008.prev_1008;
67     }
68     System.out.println("-----");
69 }

```

Fungsi ini hampir mirip dengan fungsi tampilMaju_1008, perbedaanya hanya mencetak dari node terakhir sampai node belakang

Program mencetak dari belakang karena menggunakan nilai tail dan menggunakan pointer prev untuk mencetak masing masing nilai node

```

70 // 5. cariLagu_1008() - Cari lagu berdasarkan judul
71 public void cariLagu_1008(String judul) {
72     if (head_1008 == null) {
73         System.out.println(" Playlist kosong.");
74         return;
75     }
76     Lagu_2511531008 curr_1008 = head_1008;
77     int posisi = 1;
78     while (curr_1008 != null) {
79         if (curr_1008.getJudul_1008().equalsIgnoreCase(judul)) {
80             System.out.println(" Lagu ditemukan di posisi " + posisi + "!");
81             System.out.println(" Judul : " + curr_1008.getJudul_1008());
82             System.out.println(" Penyanyi : " + curr_1008.getPenyanyi_1008());
83             return;
84         }
85         curr_1008 = curr_1008.next_1008;
86         posisi++;
87     }
88     System.out.println(" Lagu \"" + judul + "\"" tidak ditemukan.");
89 }

```

Penjelasan singkat:

Fungsi ini mencari judul lagu di dalam DLL dengan menggunakan perulangan

Jika head_1008 == null, atau playlist kosong, maka cetak "playlist kosong"

Jika ditemukan (ignore case), maka cetak judul dan penyanyi dari lagu tersebut, jika tidak, maka cetak lagu tidak ditemukan

```

90 public static void menulagu_1008() {
91     // Tampilan menu utama
92     System.out.println("=== Playlist Musik NIM: 2511531008 ===");
93     System.out.println("1. Tambah Lagu");
94     System.out.println("2. Hapus Lagu Pertama");
95     System.out.println("3. Lihat Playlist (Maju)");
96     System.out.println("4. Lihat Playlist (Mundur)");
97     System.out.println("5. Cari Lagu");
98     System.out.println("6. Keluar");
99 }

```

Penjelasan :

Method ini untuk menampilkan menu yang terdapat pada lagu

```

101 public static void main(String[] args) {
102     Scanner sc_1008 = new Scanner(System.in);
103     Musik_2511531008 playlist_1008 = new Musik_2511531008();
104     int pilihan_1008;
105     do {
106         menuLagu_1008();
107         System.out.print("Pilihan: ");
108         pilihan_1008 = sc_1008.nextInt();
109         sc_1008.nextLine(); // buang newline sisa
110         switch (pilihan_1008) {
111             case 1:
112                 // Tambah Lagu
113                 System.out.print("Judul : ");
114                 String judul_1008 = sc_1008.nextLine();
115                 System.out.print("Penyanyi : ");
116                 String penyanyi_1008 = sc_1008.nextLine();
117                 playlist_1008.tambahLagu_1008(judul_1008, penyanyi_1008);
118                 break;
119             case 2:
120                 playlist_1008.hapusLaguAwal_1008();
121                 break;
122             case 3:
123                 playlist_1008.tampilMaju_1008();
124                 break;
125             case 4:
126                 playlist_1008.tampilMundur_1008();
127                 break;
128             case 5:
129                 System.out.print("Masukkan judul yang dicari: ");
130                 String cari_1008 = sc_1008.nextLine();
131                 playlist_1008.cariLagu_1008(cari_1008);
132                 break;
133             case 6:
134                 System.out.println("Terima kasih, program selesai!");
135                 break;
136             default:
137                 System.out.println(" Pilihan tidak tersedia, coba lagi.");
138         }
139     } while (pilihan_1008 != 6);
140 }

```

Penjelasan:

Diawali dengan memilih menu dari 1-6, jika input angka 1, maka program akan menginput judul dari penyanyi.

Jika mengisi selain dari 1-6 , maka program akan mencetak “pilihan tidak tersedia, coba lagi.”

Ouput

```

=== Playlist Musik NIM: 2511531008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul : shape of you
Penyanyi : gak tau
Lagu berhasil ditambahkan!
=== Playlist Musik NIM: 2511531008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul : beauty and a beat
Penyanyi : justin bieber
Lagu berhasil ditambahkan!
=== Playlist Musik NIM: 2511531008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3

Playlist (Maju) :
-----
1. shape of you - gak tau
2. beauty and a beat - justin bieber

-----
=== Playlist Musik NIM: 2511531008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4

Playlist (Mundur) :
-----
1. beauty and a beat - justin bieber
2. shape of you - gak tau
-----
=== Playlist Musik NIM: 2511531008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul yang dicari: balab
Lagu "balab" tidak ditemukan.
=== Playlist Musik NIM: 2511531008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu "shape of you" berhasil dihapus.

```